

# TECNOLOGIA EM DESENVOLVIMENTO DE SOFTWARE MULTIPLATAFORMA







Técnica de Programação I

Profº Luiz Cláudio

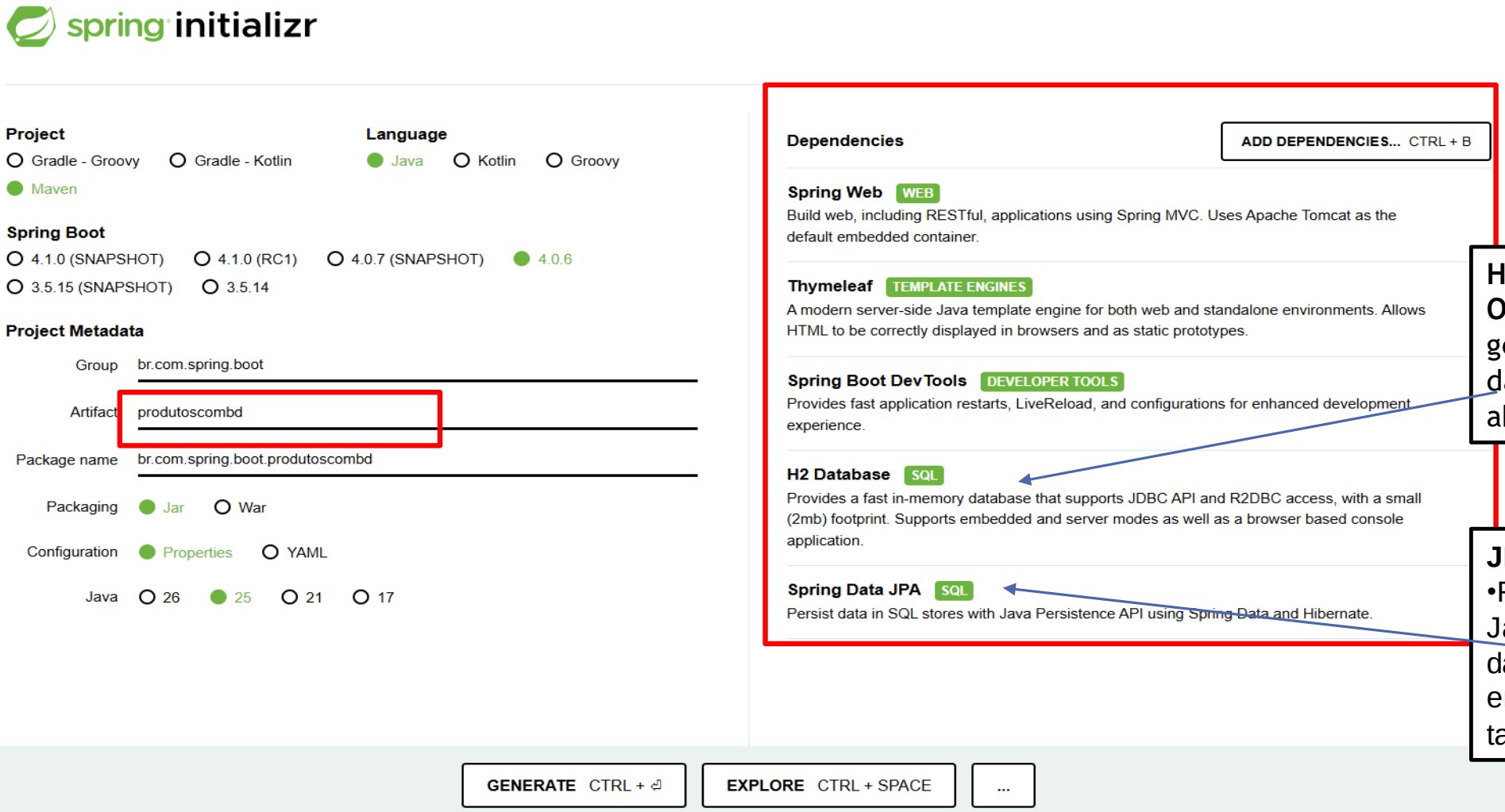
Fatec Registro

CPS Centro Paula Souza

SÃO PAULO GOVERNO DO ESTADO

# PROJETO FORMULÁRIO COM SPRING BOOT – SALVANDO DADOS COM H2 - JPA

Entre no site https://start.spring.io/, para criar o projeto spring boot.Depois de adicionar as dependências(web,dev,thyme) , clique em generate , para criar e baixar seu projeto

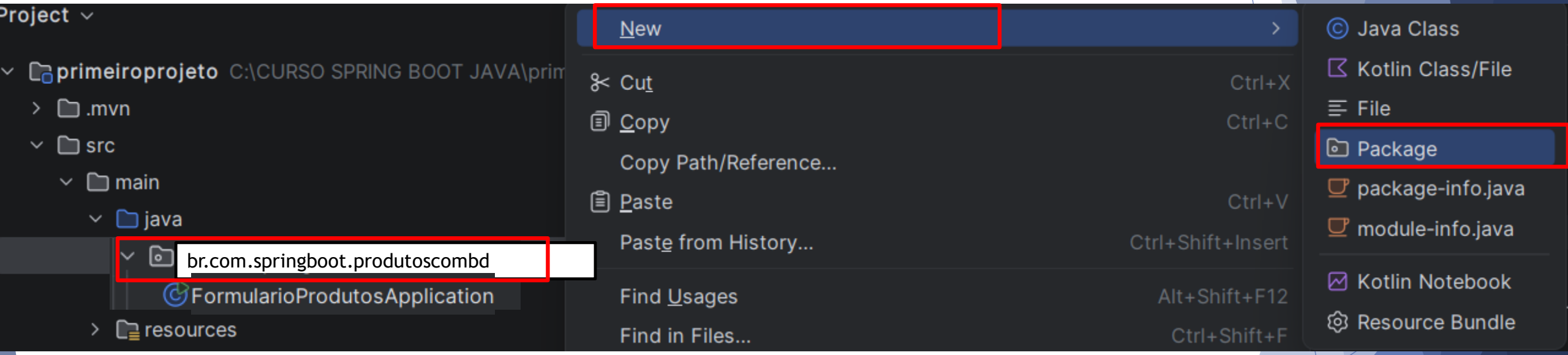


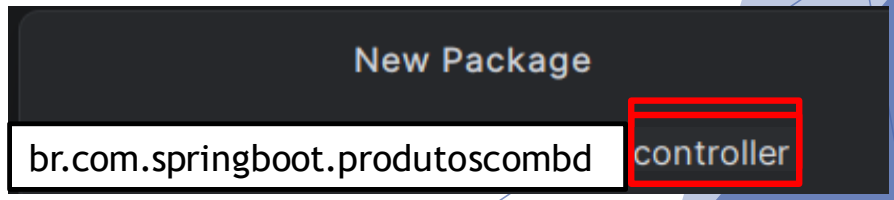

**H2 Database**
**O que é:** Um sistema de gerenciamento de banco de dados SQL rápido e de código aberto.

**JPA (Java Persistence API)**
•Faz a ponte entre objetos Java e tabelas de banco de dados relacionais, mapeando entidades Java (`@Entity`) para tabelas SQL.

# PROJETO FORMULÁRIO COM SPRING BOOT – SALVANDO DADOS COM H2 - JPA

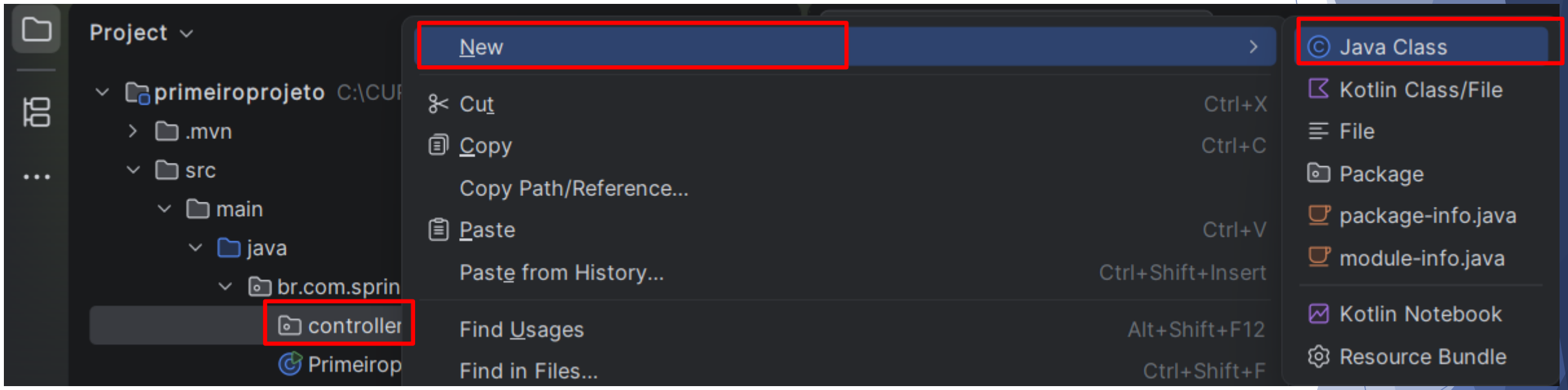
Para iniciarmos , vamos criar um pacote controller , pressione o botão direito , sobre o nome do pacote do projeto , selecione New->Package e digite controller

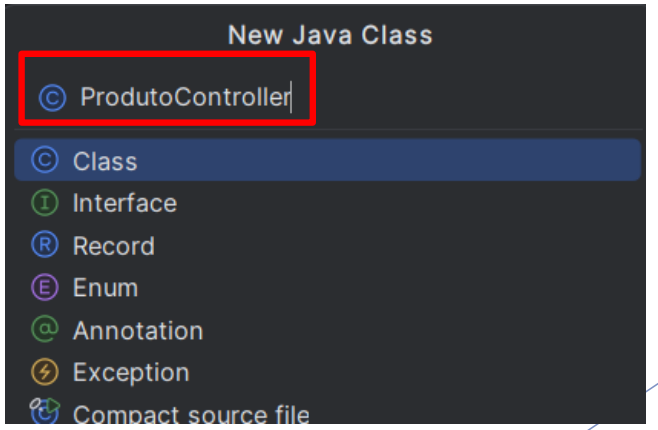


# PROJETO FORMULÁRIO COM SPRING BOOT – SALVANDO DADOS COM H2 - JPA

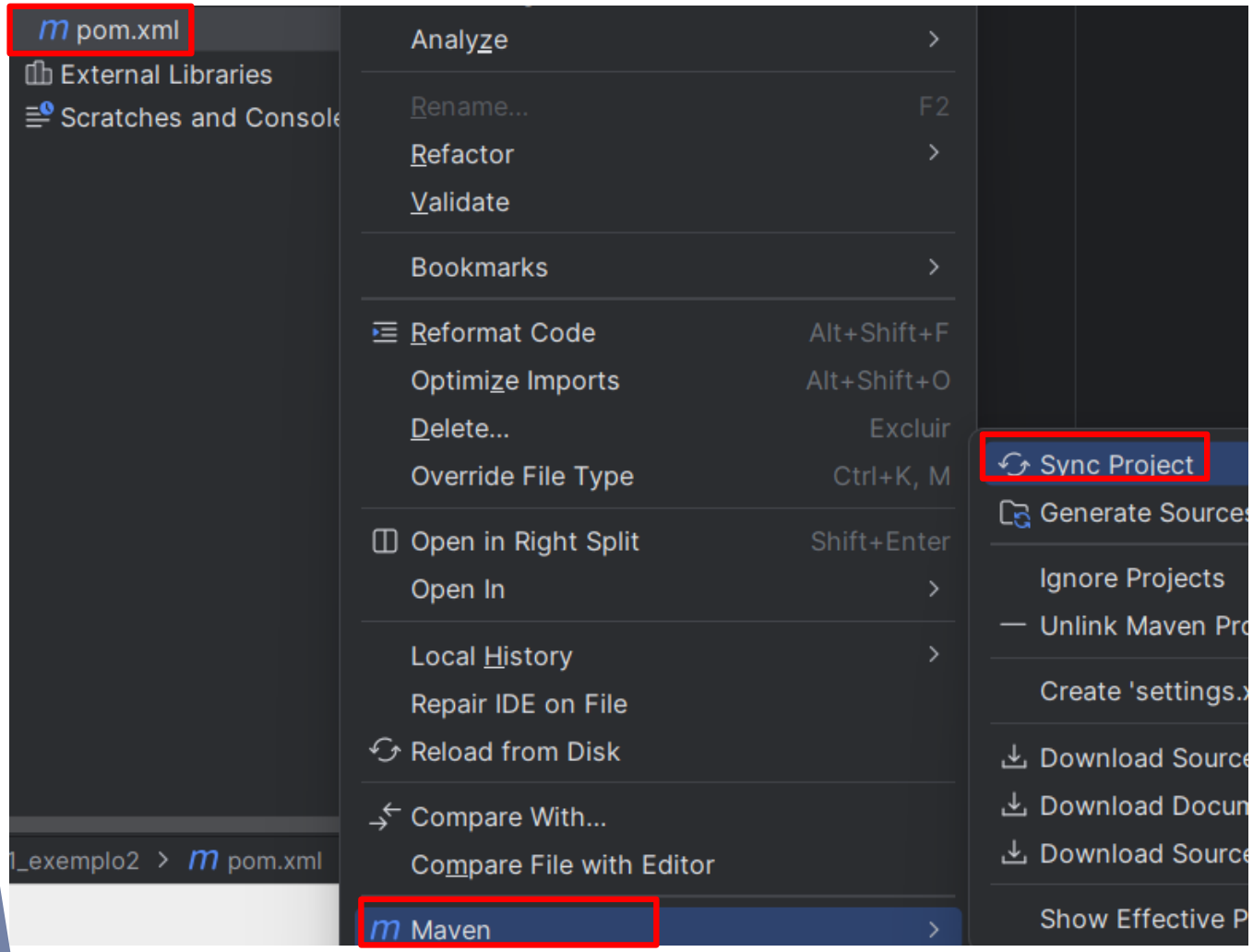
Depois crie uma classe dentro do controller , mesmo processo pressione botão direito , no botão controller , New->Java Class , coloque o nome da classe, com as Iniciais em Maiúsculo , ProdutoController.



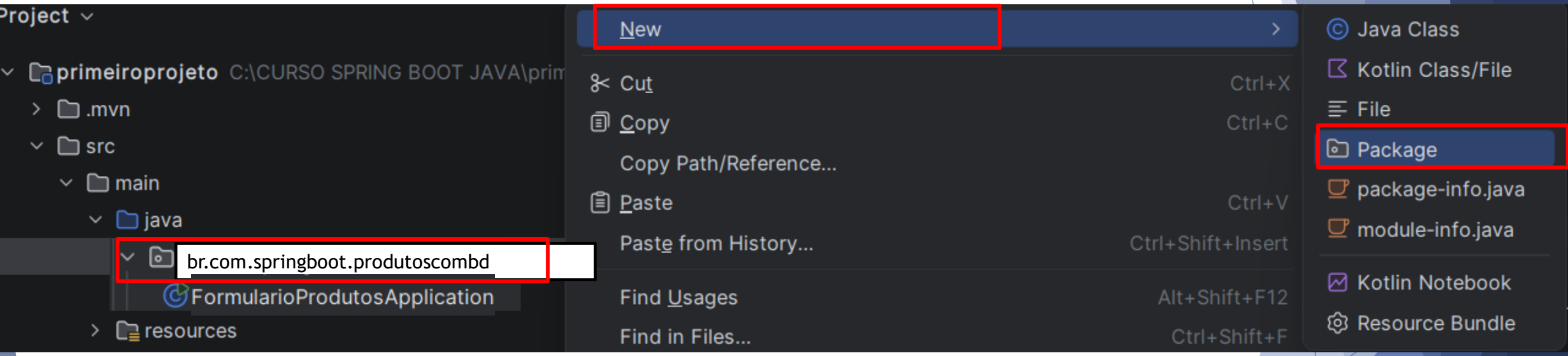
# PROJETO FORMULÁRIO COM SPRING BOOT – SALVANDO DADOS COM H2 - JPA

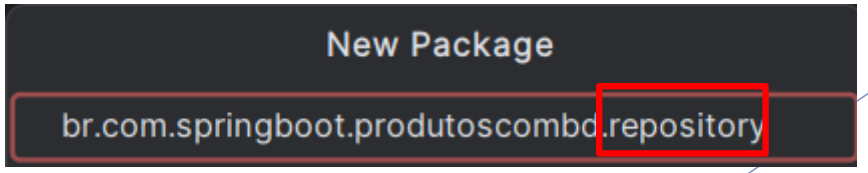
Para adicionar o maven no projeto , clique com botão direito no arquivo pom.xml , Maven->Sync Project



# PROJETO FORMULÁRIO COM SPRING BOOT – SALVANDO DADOS COM H2 - JPA

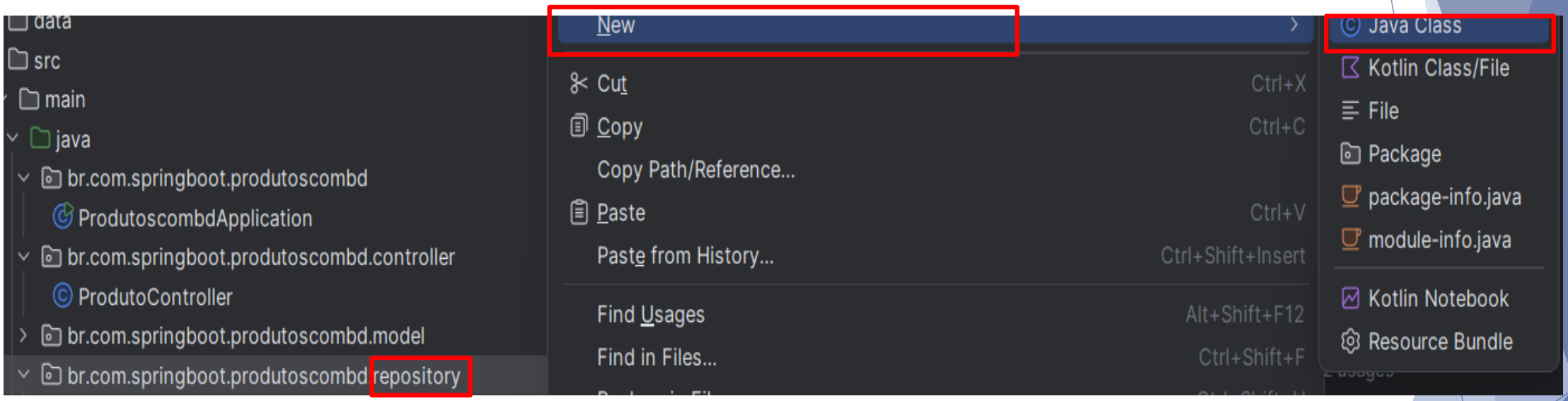
Para iniciarmos , vamos criar um pacote controller , pressione o botão direito , sobre o nome do pacote do projeto , selecione New->Package e digite repository

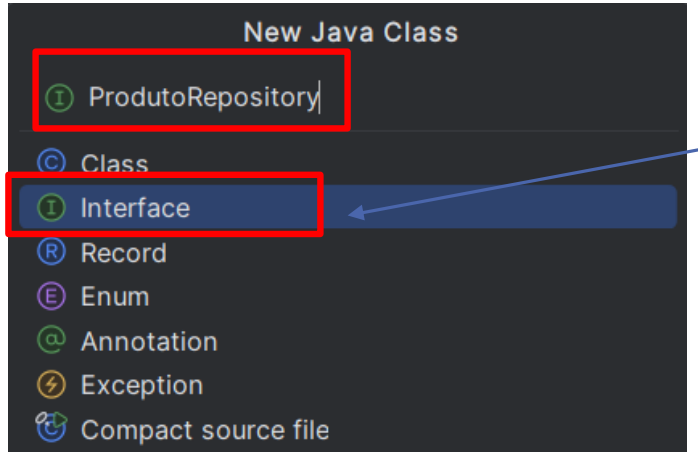


# PROJETO FORMULÁRIO COM SPRING BOOT – SALVANDO DADOS COM H2 - JPA

Depois crie uma classe do tipo Interface , dentro do repository , mesmo processo pressione botão direito , no pacote do repository, New->Java Class , escolha a interface e coloque o nome da classe, com as Iniciais em Maiúsculo , ProdutoRepository




Uma **interface** em Java é um estrutura que define um comportamento, contendo apenas assinaturas de métodos (sem corpo) ,que as classes devem obrigatoriamente implementar

# PROJETO FORMULÁRIO COM SPRING BOOT – SALVANDO DADOS COM H2 - JPA

Defina em ProdutoRepository a herança do JPA , identificando a classe Produto , com chave primaria de números inteiros

```
package br.com.springboot.produtoscombd.repository;

import br.com.springboot.produtoscombd.model.Produto;
import org.springframework.data.jpa.repository.JpaRepository;


public interface ProdutoRepository extends JpaRepository<Produto, Integer> {

}
```

**extends JpaRepository**

A palavra extends significa que sua interface está herdando funcionalidades da interface JpaRepository.

O JpaRepository já possui vários métodos prontos. Assim não precisa programar manualmente SQL. (como insert, select, update...

**<Produto, Integer>**

Produto Entidade que será manipulada
Tipo da chave primária (ID)

## PROJETO FORMULÁRIO COM SPRING BOOT – SALVANDO DADOS COM H2 - JPA



No pacote Controller no arquivo ProdutoController . Defina a anotação @Controller, para definir que a class será do tipo Controller e será criada automaticamente, e também automaticamente a importação será definida, crie um objeto ProdutoRepository que defina a Interface ProdutoRepository e defina um método ProdutoController

```
package br.com.springboot.produtoscombd.controller;

import br.com.springboot.produtoscombd.model.Produto;
import br.com.springboot.produtoscombd.repository.ProdutoRepository;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;

@Controller
public class ProdutoController {

    private ProdutoRepository produtoRepository;  2 usages

    public ProdutoController(ProdutoRepository produtoRepository) {

        this.produtoRepository = produtoRepository;
    }
```

# PROJETO FORMULÁRIO COM SPRING BOOT – SALVANDO DADOS COM H2 - JPA

Ainda no Controller que terá com uma retorno uma String, na qual irá retornar uma View(formulário), que iremos construir, e coloque uma anotação GetMapping , para passar os dados a página HTML. Também um método salvar com assinatura PostMapping

```
        this.produtoRepository = produtoRepository;
    }

    @GetMapping("/formulario")
    public String formulario(Model model){

        model.addAttribute( attributeName: "produto", new Produto());

        return "formulario";
    }

    @PostMapping("/salvar")
    public String salvar(Produto produto){

        produtoRepository.save(produto);

        return "redirect:/formulario";
    }
```

O @GetMapping é o que chamamos de "anotação composta". Ele já vem pré-configurado para aceitar apenas requisições do tipo GET. Neste caso será carregado no localhost:8080/formulario

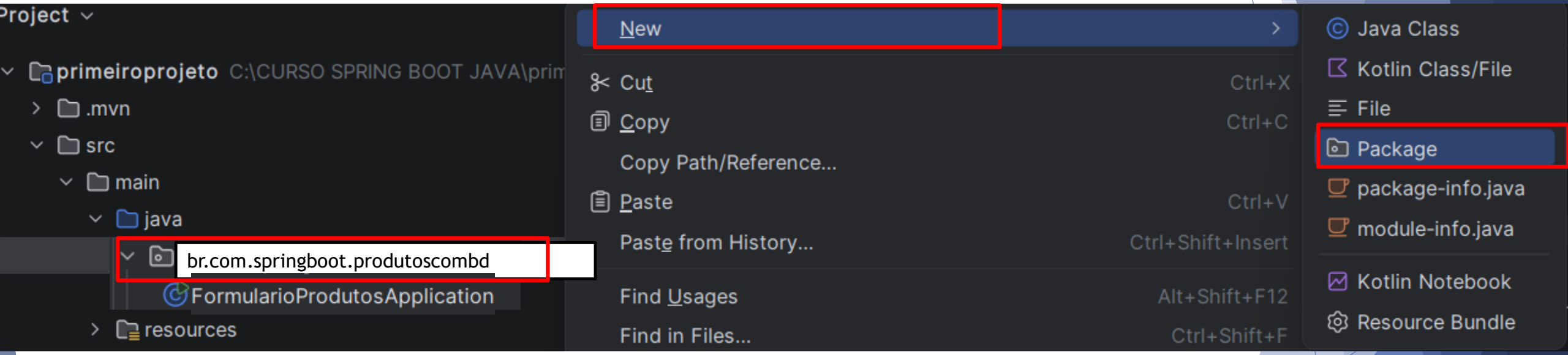
O @PostMapping usada para indicar que um método do controller responderá a **requisições HTTP do tipo POST**. serve para **receber dados enviados** pelo formulário. Depois será chamado a pagina salvar para armazenar no banco de dados

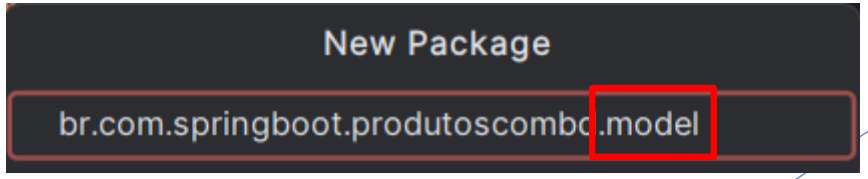
Usa o repositório e chama o método padrão do JPA save , para inserir dados no BD.

Redirect evita que o formulário faça duplicação de valores, neste caso ele faz uma nova requisição do post

# PROJETO FORMULÁRIO COM SPRING BOOT – SALVANDO DADOS COM H2 - JPA

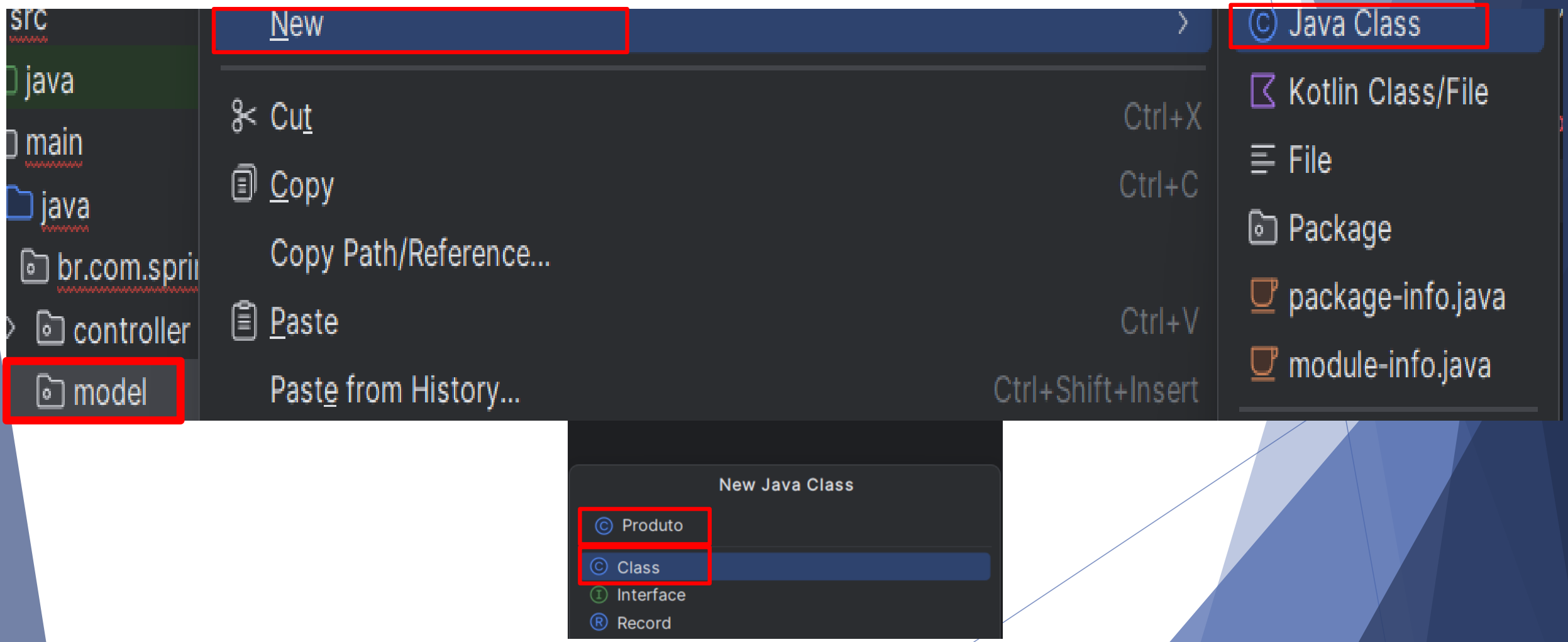
Vamos criar a classe Produto, pressione o botão direito , sobre o nome do pacote do projeto , selecione New->Package e digite model



# PROJETO FORMULÁRIO COM SPRING BOOT – SALVANDO DADOS COM H2 - JPA

Vamos criar a classe Produto, pressione o botão direito , sobre o nome do pacote model , selecione New->Class  e digite Produto



# PROJETO FORMULÁRIO COM SPRING BOOT – SALVANDO DADOS COM H2 - JPA

Na Classe Produto , importe as anotações @Entity , que representa a entidade Produto e @Table que será criada no banco de dados com nome de "produto"

```java
package br.com.springboot.produtoscombd.model;

import jakarta.persistence.*;

@Entity
@Table(name = "produto")
public class Produto {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    private String nome;  2 usages

    private String descricao;  2 usages

    private Double preco;  2 usages
```

Jakarta, permite usar as anotações Entity , Table e ID

Define a chave primária

Define que a chave será auto_increment

Crie os atributos private id,nome,descrição,preco



# PROJETO FORMULÁRIO COM SPRING BOOT – SALVANDO DADOS COM H2 - JPA

Após criar faça o getter e setter dos atributos da classe Produto, após declarar os atributos , clique botão direito generate -> getter and setter -> selecione todos os atributos

Atributos depois do encapsulamento getter e setter

```java
package br.com.springboot.produtoscombd.model;

import jakarta.persistence.*;

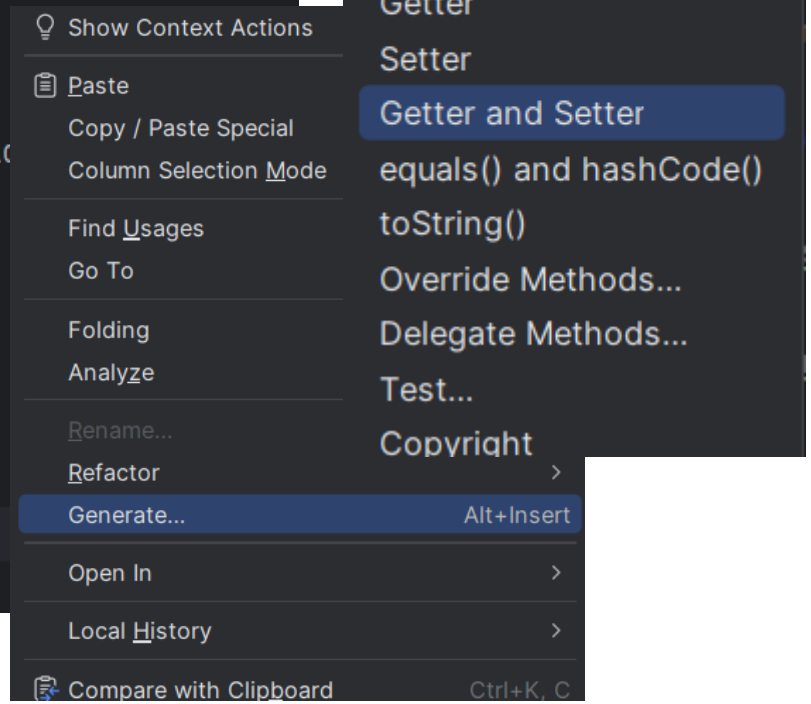
@Entity
@Table(name = "produto")
public class Produto {

    @Id
    @GeneratedValue(strategy = Generatic
    private Integer id;

    private String nome;  2 usages

    private String descricao;  2 usages

    private Double preco;  2 usages
```




```java
public Integer getId() { return id; }

public void setId(Integer id) { this.id = id; }

public String getNome() { return nome; }

public void setNome(String nome) { this.nome = nome; }

public String getDescricao() { return descricao; }

public void setDescricao(String descricao) { this.descricao = descricao; }

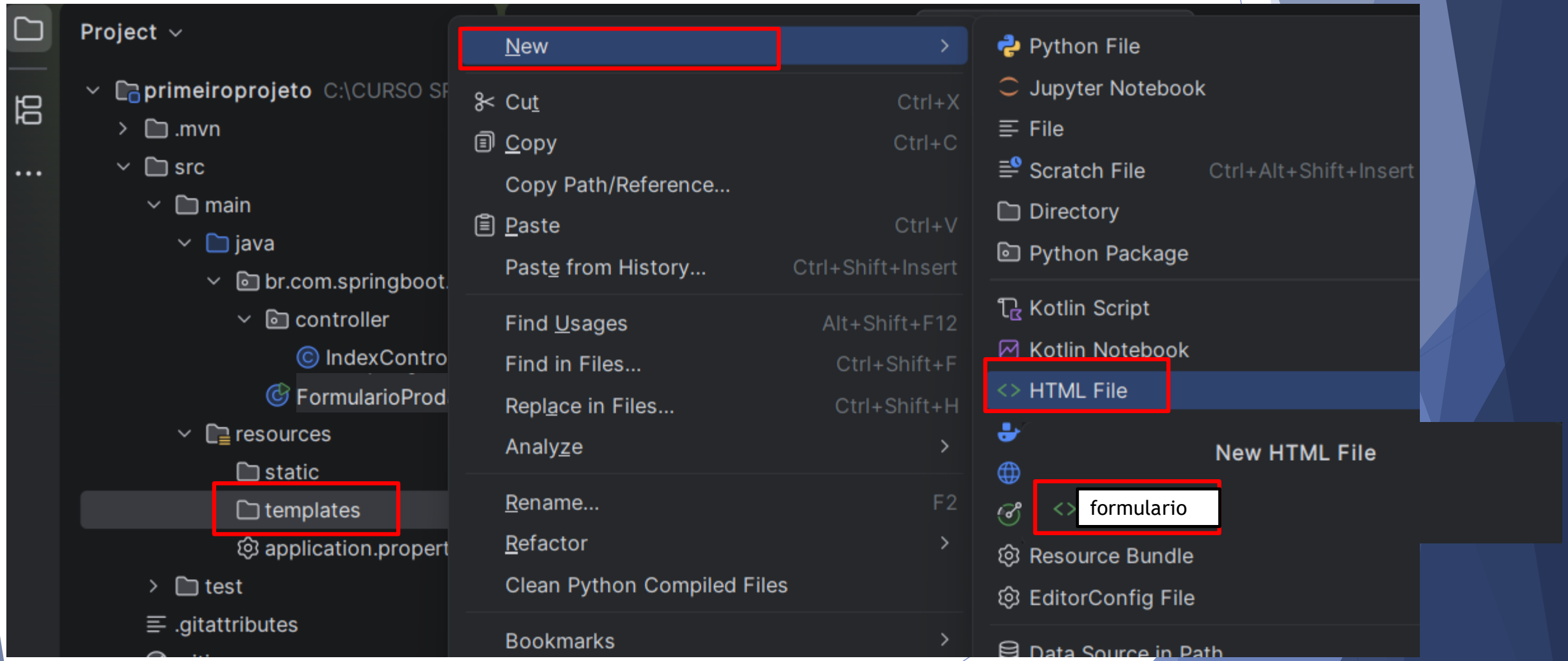
public Double getPreco() { return preco; }

public void setPreco(Double preco) { this.preco = preco; }
```



# PROJETO FORMULÁRIO COM SPRING BOOT – SALVANDO DADOS COM H2 - JPA

No diretório resources, clique com botão direito em templates , selecione New->HTML File , crie um arquivo index.html



# PROJETO FORMULÁRIO COM SPRING BOOT – SALVANDO DADOS COM H2 - JPA

No arquivo formulario.html será criado um formulário para  digitar os dados do produto e depois ao clicar e cadastrar será chamado o método salvar

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="UTF-8">
  <title>Cadastro de Produtos</title>

  <style>
    body{
        font-family: Arial;
        background-color: #a2a2a2;
    }
    .container{
        width: 500px;
        margin: auto;
        background: white;
        padding: 20px;
        margin-top: 50px;
        border-radius: 10px;
    }
    input{
        width: 475px;
        padding: 10px;
        margin-bottom: 10px;
    }
    button{
        width: 100%;
        padding: 10px;
        background: blue;
        color: white;
        border: none;
    }
  </style>
```

O thymeleaf deve ser declarado aqui para que os dados sejam recebidos do Controller

Defina os estilos do formulário

# PROJETO FORMULÁRIO COM SPRING BOOT – SALVANDO DADOS COM H2 - JPA

Construa o formulário pegando os valores dos objetos da classe model Produto

```
  </style>

</head>
<body>
<div class="container">

  <h2>Cadastro de Produtos</h2>

  <form th:action="@{/salvar}" th:object="${produto}" method="post">

    <input type="text"
           th:field="*{nome}"
           placeholder="Nome do produto">
    <input type="number"
           step="0.01"
           th:field="*{preco}"
           placeholder="Preço">
    <input type="text"
           th:field="*{descricao}"
           placeholder="Descrição">
    <button type="submit">
      Salvar
    </button>
  </form>
</div>
</body>
</html>
```

define para onde os dados do formulário serão enviados.Neste caso a página salvar

define o objeto principal do formulário.
O objeto produto veio do controller:

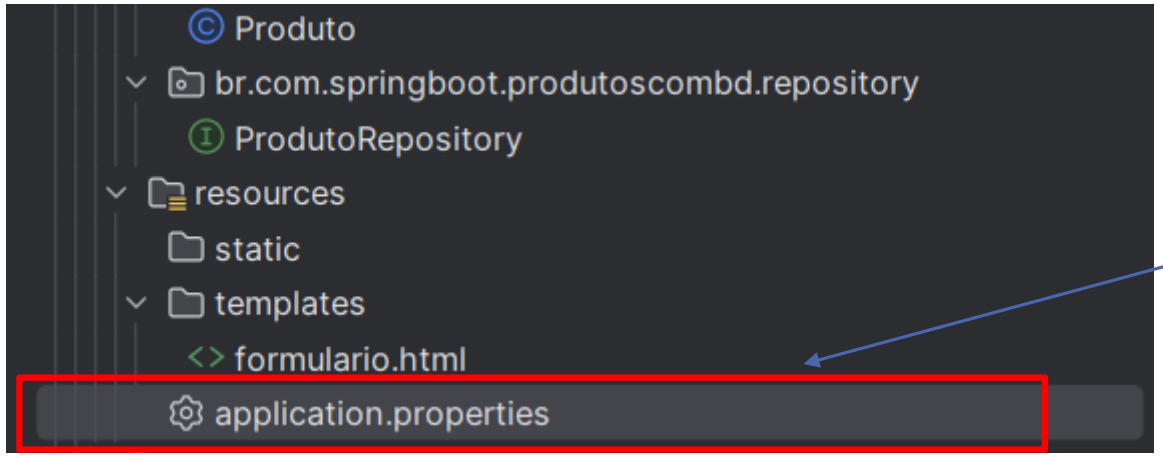
model.addAttribute("produto", new Produto());

*{ } acessa o atributo, pelo objeto(produto): produto.preco

Botão que confirma o cadastro(submit)chama salvar(página defina no action)

# PROJETO FORMULÁRIO COM SPRING BOOT – SALVANDO DADOS COM H2 - JPA

Clique no arquivo aplicattion.properties para definir o acesso ao Banco de Dados h2




```
spring.application.name=produtoscombd


spring.datasource.url=jdbc:h2:file:./data/produtos
spring.datasource.username=sa
spring.datasource.password=password
spring.datasource.driver-class-name=org.h2.Driver

spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto=update

spring.h2.console.enabled=true
spring.h2.console.path=/bancoproduto
```

Clique neste arquivo abaixo do template

Define onde está o banco de dados. E conecta ao Banco de dados

Define  o nome do usuário e a senha

Define a classe Java responsável por conectar ao H2.

Define o SQL para criar o banco de dados.

Para a tabela atualizar automaticamente

Ative a interface da web h2 automaticamente

Define o caminho para acessa o h2 na web http://localhost:8080/bancoproduto

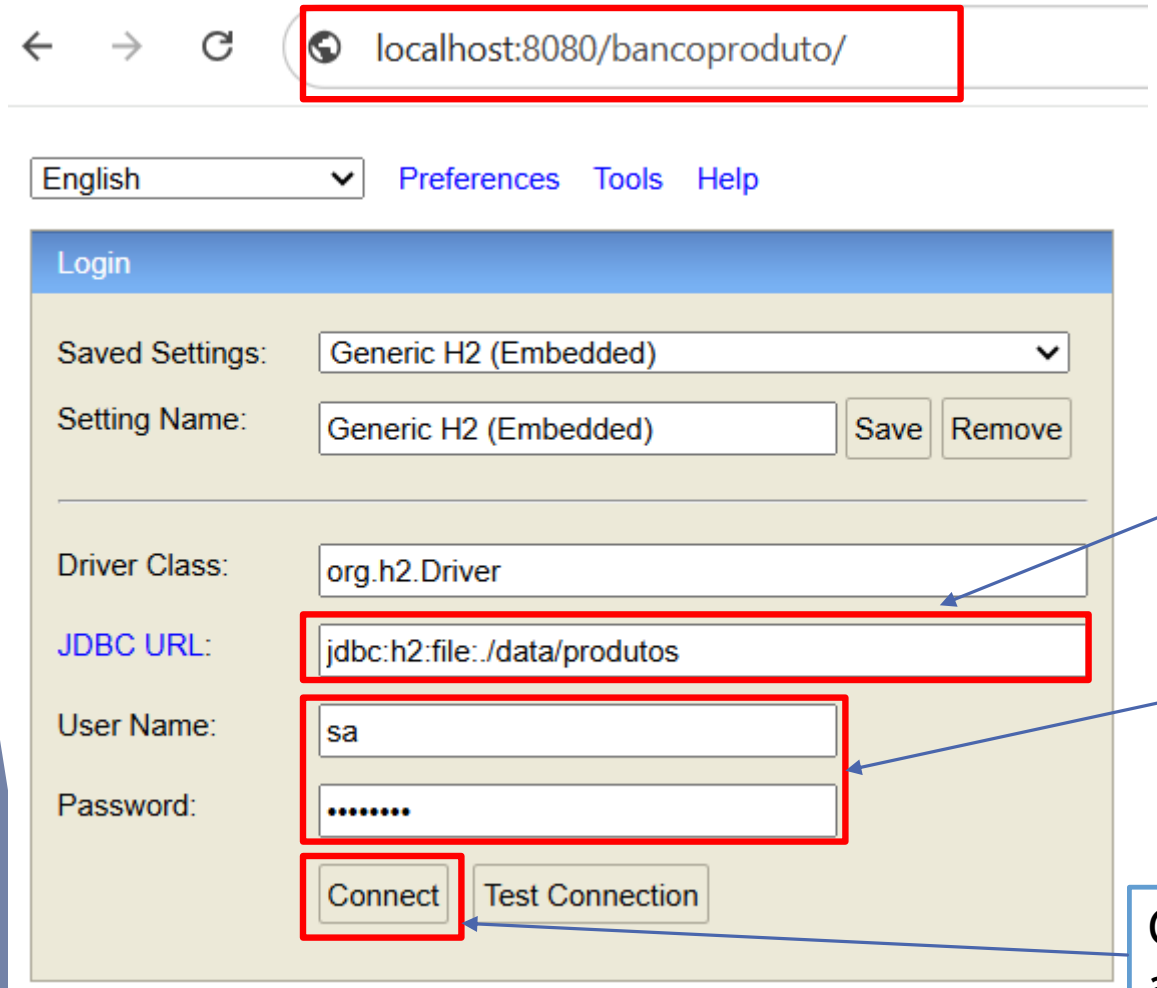
# PROJETO FORMULÁRIO COM SPRING BOOT – SALVANDO DADOS COM H2 - JPA

Para executar. Volte na classe Principal , clique no botão Run -> Run ProdutoscombdApplication



# PROJETO FORMULÁRIO COM SPRING BOOT – SALVANDO DADOS COM H2 - JPA

Abra o navegador com a URL : localhost:8080/bancoproduto



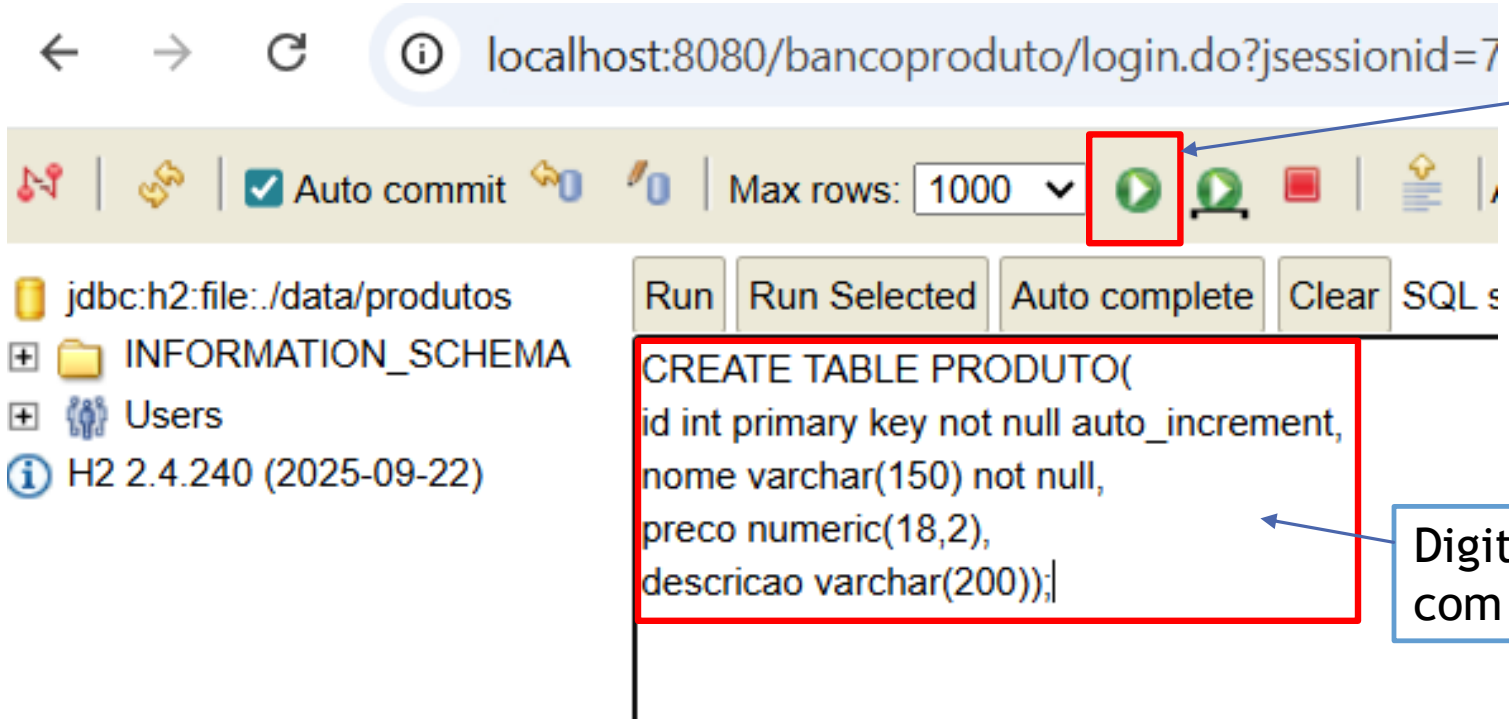

Coloque aqui o caminho do banco de dados , como foi colocado no arquivo application.properties

Coloque o usuário e a senha definidos no arquivo application.properties

Clique em Connect , para conectar ao Banco a interface H2

# PROJETO FORMULÁRIO COM SPRING BOOT – SALVANDO DADOS COM H2 - JPA

Na interface do h2 , caso a tabela produto não crie automaticamente, faça o código SQL e clique em executar



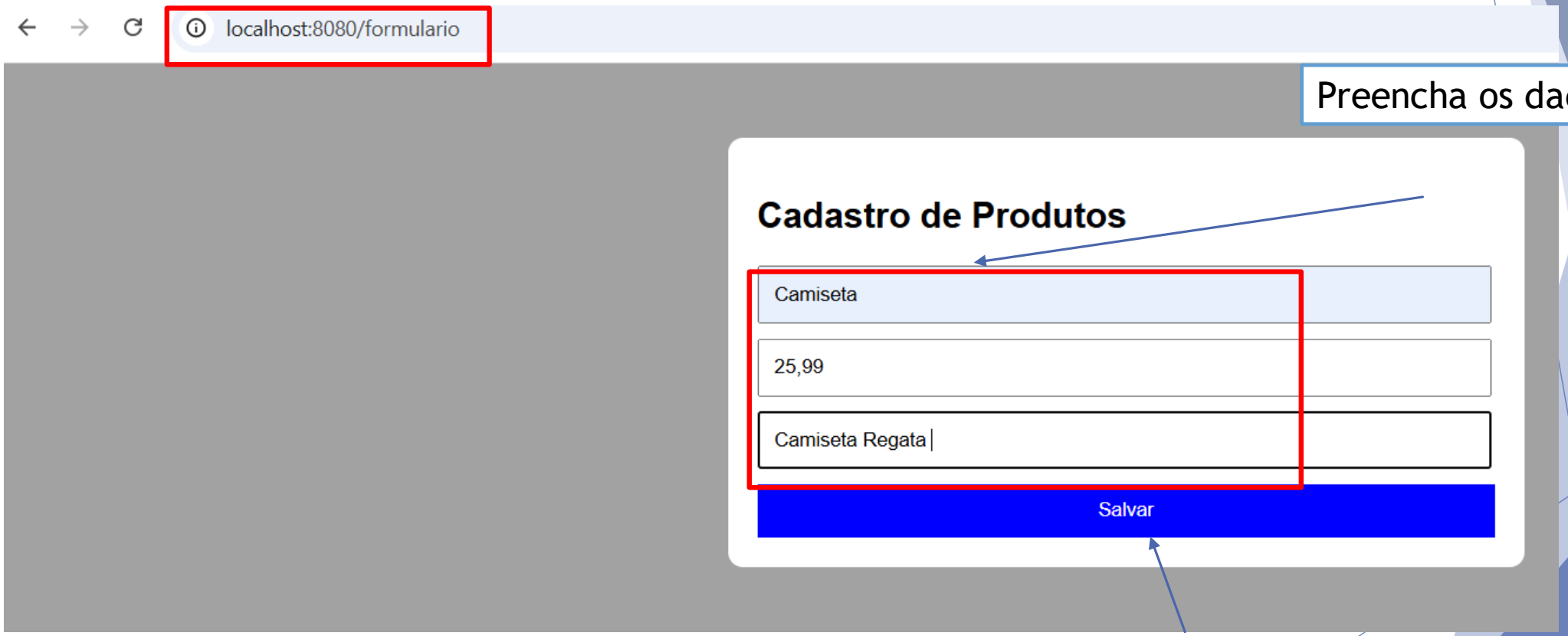

Clique em executar , após escrever o código sql

Digite o código SQL , crie a tabela produto, com os atributos id, nome ,preco,descricao

# PROJETO FORMULÁRIO COM SPRING BOOT – SALVANDO DADOS COM H2 - JPA

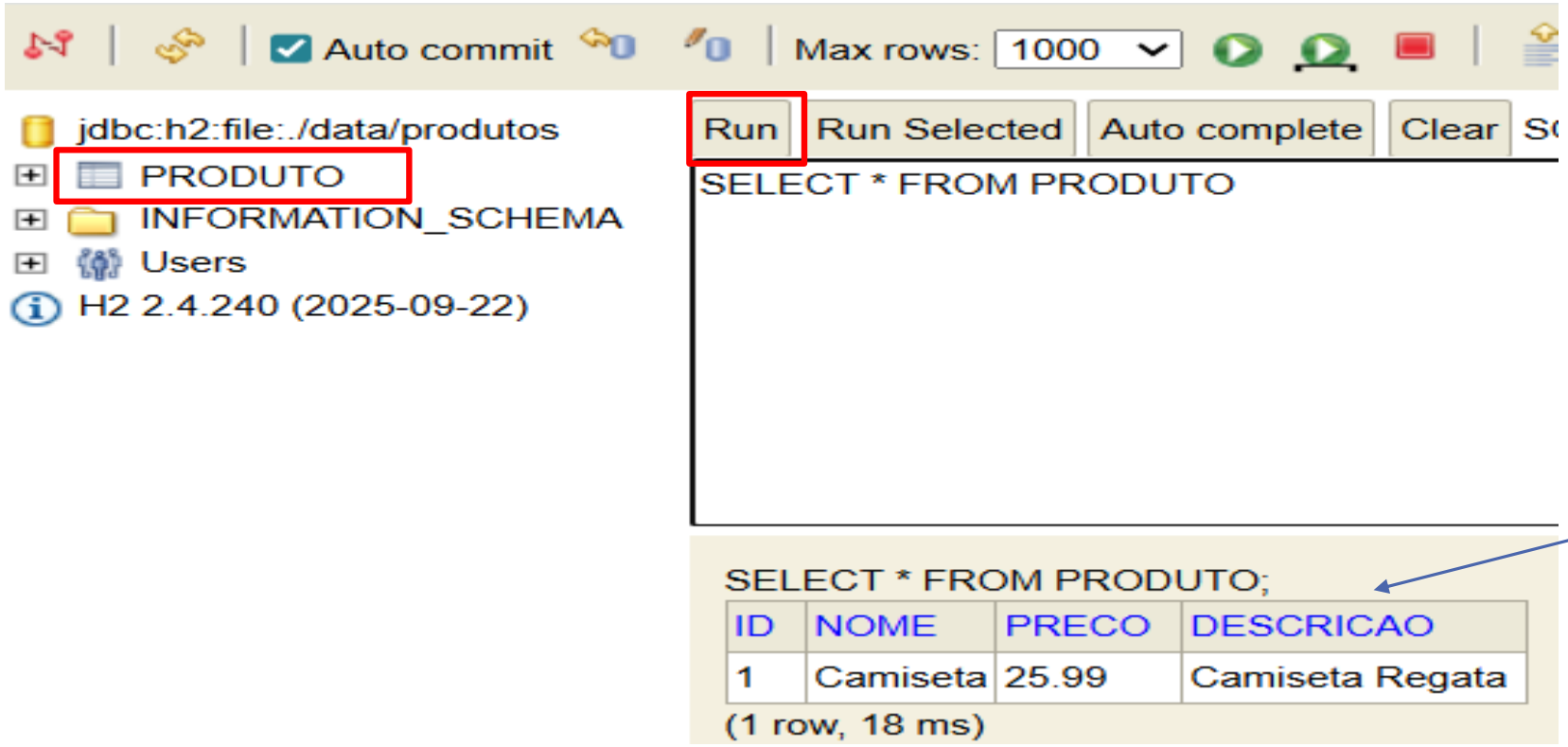
Para Executar o formulário e enviar os dados ao BD, digite a url:

http://localhost:8080/formulario



# PROJETO FORMULÁRIO COM SPRING BOOT – SALVANDO DADOS COM H2 - JPA

Acesse o H2 com a URL URL : localhost:8080/bancoproduto , e verifique se os dados foram cadastrados, clique no nome da tabela, em seguida clique em Run




Dados Cadastrados no Banco de Dados